Trabajo Práctico N° 2

Redes Convolucionales-Recurrentes y Aprendizaje por Refuerzo

Aprendizaje Automático II

Tecnicatura Universitaria en Inteligencia Artificial

FCEIA - UNR

Introducción

Este trabajo práctico se compone de cuatro ejercicios. El **Ejercicio 1** aborda el modelado de secuencias mediante una **Red Neuronal Convolucional-Recurrente (CRNN)** aplicada al reconocimiento de comandos de voz. Los **Ejercicios 2, 3 y 4** abordan los dos paradigmas centrales del aprendizaje por refuerzo: **Q-Learning clásico** y **Deep Q-Learning (DQN)**, junto con un análisis teórico comparativo entre ambos.

Entregables

- **Jupyter Notebooks** (`.ipynb`): deben ejecutarse completamente de principio a fin sin errores, con celdas de código y celdas de texto (Markdown) que expliquen las decisiones tomadas, los experimentos realizados y el análisis de resultados. La organización de los notebooks se detalla en la sección de Modalidad de Entrega.
- **Informe** (`.pdf`): contiene la descripción del procedimiento seguido, las decisiones de diseño y las respuestas a todas las consignas de análisis. No incluir bloques de código extensos; solo fragmentos breves cuando sean necesarios para justificar decisiones de implementación.

Consideraciones Generales

- Se recomienda encarecidamente guardar **checkpoints** durante el entrenamiento y aprovechar la funcionalidad de **resume** para retomar desde el último checkpoint en caso de interrupción.
- **Reproducibilidad:** Fijar una semilla aleatoria al inicio de cada experimento para que los resultados sean reproducibles. Esto incluye las semillas de Python, NumPy y el framework de deep learning utilizado:

```python
import random, numpy as np, torch

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(42)
```

En los entornos de Gymnasium (Ejercicios 2 y 3), además, pasar la semilla al primer reset del entorno y al espacio de acciones:

```python
obs, info = env.reset(seed=42)
env.action_space.seed(42)
```

- **Checkpoints y reanudación:** se recomienda guardar el estado del modelo periódicamente para poder retomar el entrenamiento en caso de interrupción:

```python
# Guardar checkpoint
torch.save({
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': loss,
}, 'checkpoint_epoch_{}.pth'.format(epoch))

# Retomar desde checkpoint
checkpoint = torch.load('checkpoint_epoch_N.pth')
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
start_epoch = checkpoint['epoch'] + 1
```

- **División del dataset (Ejercicio 1):** Siempre trabajar con tres conjuntos separados: **train** (entrenamiento), **val** (validación) y **test** (evaluación final). El conjunto de train es el único que el modelo ve durante el entrenamiento. El conjunto de val se usa para monitorear el entrenamiento, comparar experimentos y tomar decisiones de diseño (arquitectura, hiperparámetros). El conjunto de test se usa **una sola vez**, al final, para reportar las métricas definitivas. Usar val como criterio de selección y reportar test evita el sobreajuste implícito al conjunto de evaluación.

1. Ejercicio 1 — Reconocimiento de Comandos de Voz con CRNN

Dataset Drive: Google Speech Commands v2 - Drive

Dataset Kaggle: Google Speech Commands v2 - Kaggle

Clases sugeridas (10): `yes`, `no`, `up`, `down`, `left`, `right`, `on`, `off`, `stop`, `go`.

Descripción

En este ejercicio deben construir una **Red Neuronal Convolucional-Recurrente (CRNN)** para reconocer comandos de voz a partir de grabaciones de audio de un segundo. El dataset contiene grabaciones pronunciadas por muchos hablantes distintos en condiciones acústicas reales (ruido de fondo, variaciones de micrófono), lo que hace que el modelado temporal de la RNN tenga valor real: comandos como `down` o `stop` son secuencias fonéticas con estructura temporal que una red sin memoria no puede capturar correctamente.

La arquitectura sigue un esquema **many-to-one**: la CNN procesa cada ventana temporal del espectrograma de forma independiente y produce un vector de características; la secuencia de esos vectores entra a la RNN, que modela cómo los patrones acústicos evolucionan en el tiempo; el último estado oculto se clasifica con una capa densa.

Reducción del problema: *Si el entrenamiento completo resulta demasiado pesado para los recursos disponibles, pueden trabajar con un subconjunto de* ***4 a 6 clases*** *(ej. `yes`, `no`, `stop`, `go`). Deben justificar la selección en el informe. Se recomienda guardar* ***checkpoints*** *regularmente para poder retomar el entrenamiento ante interrupciones.*

Pipeline

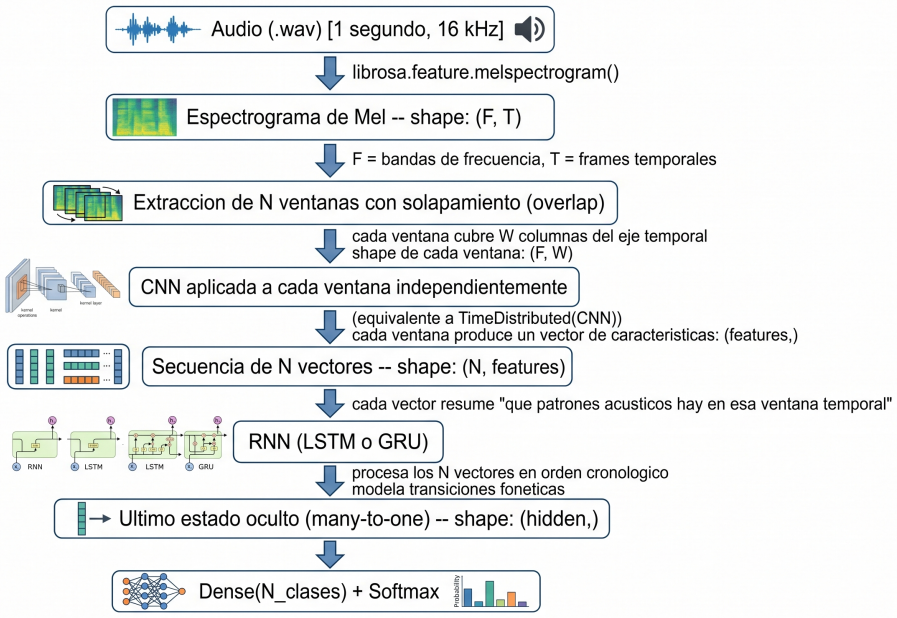


Figura 1: Pipeline propuesto para la arquitectura CRNN, desde el audio de entrada hasta la clasificación final.

1.1. Preprocesamiento: Del audio al espectrograma de Mel

Los espectrogramas de Mel representan la energía de la señal de audio en función del tiempo (eje horizontal) y la frecuencia en escala Mel (eje vertical). La escala Mel comprime las frecuencias altas y expande las bajas, imitando la percepción auditiva humana. El color representa la intensidad de energía en decibeles (dB): colores cálidos indican alta energía, colores fríos indican baja energía o silencio.

```python
import librosa
import numpy as np

def audio_a_mel(path, sr=16000, n_mels=64, n_fft=1024, hop_length=512):
    y, sr = librosa.load(path, sr=sr)            # cargar y resamplear a 16 kHz
    mel = librosa.feature.melspectrogram(
        y=y, sr=sr, n_mels=n_mels,
        n_fft=n_fft, hop_length=hop_length
    )
    mel_db = librosa.power_to_db(mel, ref=np.max)  # escala logaritmica (dB)
    return mel_db  # shape: (F, T) = (n_mels, T)
```

Pasos obligatorios

- Fijar `sr=16000` (frecuencia de muestreo del dataset).
- Asegurarse de que todos los espectrogramas tengan la **misma longitud temporal** T (padding con ceros al final o truncado).

- **Normalizar** (media 0, desvío 1) calculando estadísticas **solo sobre el conjunto de entrenamiento** y aplicándolas a validación y test.

Visualización y análisis de espectrogramas (obligatorio)

Antes de entrenar, graficar al menos **8 espectrogramas**: 4 instancias de la misma clase y 4 de clases distintas. Para cada uno, describir explícitamente:

- **Eje horizontal (tiempo):** ¿Cuánto dura la señal útil? ¿Hay silencio al inicio o al final?
- **Eje vertical (frecuencia Mel):** ¿En qué rango de frecuencias se concentra la energía del comando?
- **Color (energía en dB):** ¿Dónde están las zonas de alta energía? ¿Qué representan las bandas horizontales brillantes?

Luego responder: ¿pueden distinguir visualmente espectrogramas de la misma clase entre sí? ¿Y entre clases distintas (ej. `yes` vs `no`, `up` vs `down`)? ¿Qué diferencias observan en la distribución temporal y frecuencial de la energía?

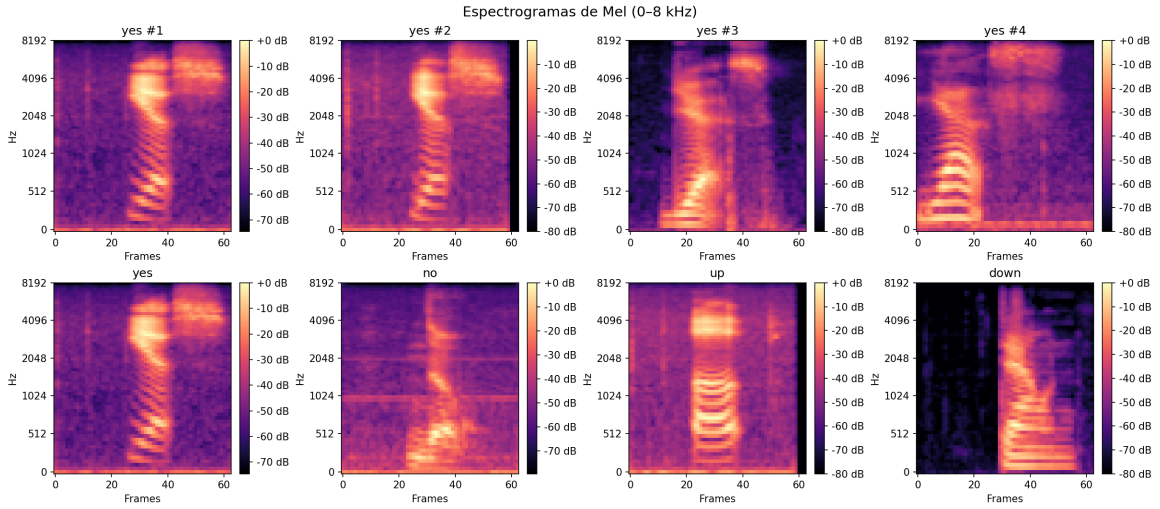


Figura 2: Panel de espectrogramas de Mel utilizado para analizar variabilidad intra-clase y diferencias entre clases.

1.2. Segmentación con ventana deslizante

Antes de pasar el espectrograma a la CNN, se divide en **ventanas temporales con solapamiento**. Cada ventana cubre W columnas del eje temporal y representa una región acústica del audio. El solapamiento garantiza que las transiciones entre sílabas no queden partidas entre dos ventanas sin contexto compartido.

```python
def extraer_ventanas(mel, window_size=16, hop=8):
    """
    mel: array (F, T)
    window_size: ancho de cada ventana en frames temporales
    hop: desplazamiento entre ventanas (hop < window_size -> solapamiento)

    Retorna: array (N, F, window_size) con N ventanas
    """
```

```
    F, T = mel.shape
    ventanas = []
    for start in range(0, T - window_size + 1, hop):
        ventana = mel[:, start : start + window_size]  # (F, W)
        ventanas.append(ventana)
    return np.stack(ventanas)  # (N, F, W)
```

El overlap entre ventanas consecutivas es `window_size - hop`. Con `window_size=16` y `hop=8` el solapamiento es del 50 %, lo que asegura que cada transición fonética aparezca representada en al menos dos ventanas consecutivas.

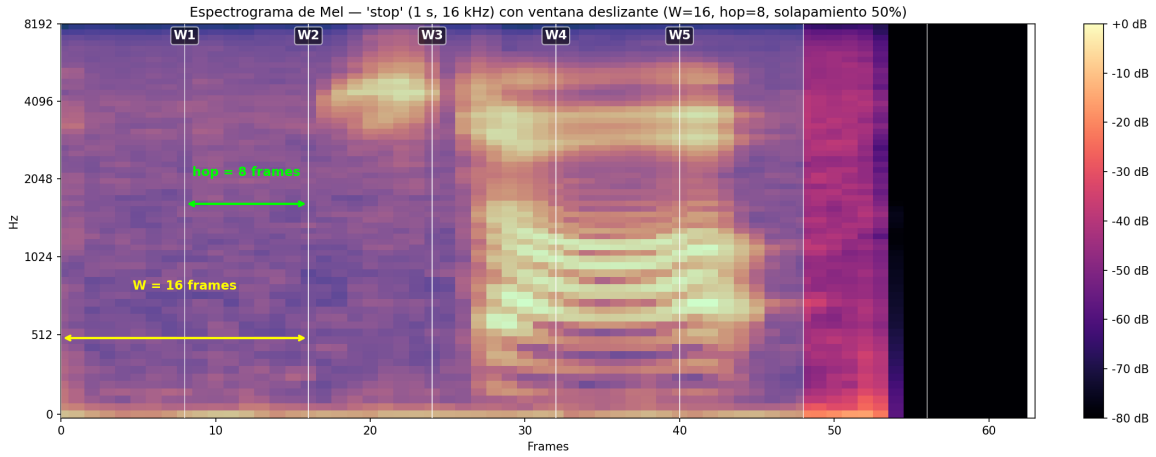


Figura 3: Ejemplo de segmentación de un espectrograma de Mel mediante ventana deslizante con solapamiento.

Nota sobre shapes: *El espectrograma tiene shape* `(F, T)` *con frecuencia en el eje 0 y tiempo en el eje 1. Cada ventana extraída tiene shape* `(F, W)`*. Al pasarla a la CNN como imagen monocanal:* `(1, F, W)`*. Es importante no confundir los ejes: la CNN debe recibir frecuencia en la dimensión de altura e índice de ventana como paso de secuencia para la RNN.*

1.3. Arquitectura CRNN

La CNN se aplica de forma **independiente a cada ventana** (equivalente a `TimeDistributed` en Keras). Su función es comprimir cada ventana `(F, W)` en un único vector de características que resume los patrones acústicos presentes en ese segmento temporal. La secuencia de esos vectores, ordenada cronológicamente, es la entrada a la RNN.

```
import torch
import torch.nn as nn

class CNNEncoder(nn.Module):
    """Procesa una sola ventana (batch, 1, F, W) -> vector (batch, features)"""
    def __init__(self, n_mels=64, window_size=16):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=(3, 3), padding=1),
            nn.ReLU(), nn.BatchNorm2d(32),
```

```
            nn.MaxPool2d(2),                        # -> (32, F/2, W/2)

            nn.Conv2d(32, 64, kernel_size=(3, 3), padding=1),
            nn.ReLU(), nn.BatchNorm2d(64),
            nn.MaxPool2d(2),                        # -> (64, F/4, W/4)
        )
        # calcular tamano de salida aplanada
        self.out_size = 64 * (n_mels // 4) * (window_size // 4)
        self.fc = nn.Linear(self.out_size, 128)

    def forward(self, x):
        # x: (batch, 1, F, W)
        x = self.cnn(x)                             # (batch, 64, F/4, W/4)
        x = x.flatten(1)                            # (batch, out_size)
        return self.fc(x)                           # (batch, 128)


class CRNN(nn.Module):
    def __init__(self, n_mels=64, window_size=16, n_classes=10,
                 rnn_hidden=128, rnn_layers=2):
        super().__init__()
        self.encoder = CNNEncoder(n_mels, window_size)

        self.rnn = nn.GRU(
            input_size=128,
            hidden_size=rnn_hidden,
            num_layers=rnn_layers,
            batch_first=True,
            bidirectional=True,
            dropout=0.3
        )
        self.fc = nn.Linear(rnn_hidden * 2, n_classes)  # *2 por
    bidireccional

    def forward(self, x):
        # x: (batch, N, 1, F, W)  -- N ventanas por muestra
        batch, N, C, F, W = x.shape
        x = x.view(batch * N, C, F, W)          # procesar todas las
    ventanas juntas
        x = self.encoder(x)                      # (batch*N, 128)
        x = x.view(batch, N, -1)                 # (batch, N, 128) --
    restaurar secuencia
        x, _ = self.rnn(x)                       # (batch, N, hidden*2)
        x = x[:, -1, :]                          # ultimo estado -> (batch,
     hidden*2)
        return self.fc(x)
```

Punto de partida: *La arquitectura anterior es una sugerencia. Se espera que optimicen tanto la CNN como la RNN para obtener la mayor precisión posible. Ver sección 1.5 para los experimentos requeridos.*

1.4. Entrenamiento

- Usar `CrossEntropyLoss` y el optimizador `Adam`.
- Implementar un scheduler de learning rate (ej. `ReduceLROnPlateau`).

- **División del dataset:** El dataset provee archivos `validation_list.txt` y `testing_list.txt` con particiones oficiales que garantizan **separación estricta por hablante**: ningún hablante del conjunto de test aparece en entrenamiento. Respetar estas particiones para que los resultados sean comparables.
- Guardar checkpoints periódicamente.

1.5. Experimentos

El objetivo es optimizar la CRNN para obtener la mayor accuracy posible en el conjunto de test. Documentar al menos **3 experimentos** en una tabla comparativa, explorando las siguientes dimensiones:

CNN (encoder de ventanas)

- Número de bloques convolucionales (profundidad).
- Número de filtros por capa.
- Tamaño de kernel.
- Dropout dentro del encoder.

Ventana deslizante

- Tamaño de ventana W (ventanas más anchas dan más contexto local a la CNN; más estrechas producen secuencias más largas para la RNN).
- Solapamiento (hop): mayor overlap → secuencia más larga, más redundancia entre tokens.

RNN

- Tipo de celda: LSTM vs. GRU.
- Número de capas y unidades ocultas.
- Bidireccional vs. unidireccional.
- Estrategia de agregación: último estado oculto vs. average pooling sobre todos los estados.

Experimento	**CNN**	**Ventana (W, hop)**	**RNN**	**Acc. Val**	**Acc. Test**	**Observaciones**
E1 — Línea base	2 bloques, 32/64 filtros	16, 8	GRU 2 capas, 128 uni.			
E2	...	...	...			
E3	...	...	...			

1.6. Qué reportar

- Curvas de loss y accuracy (train vs. validación) por época para cada experimento.

- Tabla comparativa de todos los experimentos (sección 1.5) con accuracy en validación y test.
- **Matriz de confusión** del mejor modelo sobre el conjunto de test.
- Análisis de los espectrogramas según lo indicado en la sección 1.1.
- Análisis: ¿qué comandos se confunden más? ¿Tiene sentido fonéticamente?
- Al menos 5 ejemplos de predicciones con el espectrograma, la etiqueta real y la predicción del modelo.

Aprendizaje por Refuerzo (Ejercicios 2, 3 y 4)

Los Ejercicios 2, 3 y 4 abordan los dos paradigmas centrales del aprendizaje por refuerzo: **Q-Learning clásico** y **Deep Q-Learning (DQN)**. Se utilizan dos entornos de la biblioteca **Gymnasium** (Farama Foundation) que representan casos de estudio complementarios: uno con espacio de estados discreto y finito, y otro con espacio de estados continuo.

El objetivo es que el grupo comprenda en profundidad la diferencia entre el enfoque tabular y el enfoque basado en redes neuronales, estudie el impacto de las innovaciones que introdujo el DQN original (Experience Replay y Target Network), y desarrolle criterio para seleccionar el método más adecuado según las características del problema.

Visualización del agente

Gymnasium ofrece distintos mecanismos para observar el comportamiento del agente durante y después del entrenamiento. A continuación se describen los cuatro disponibles con sus ventajas y limitaciones. En cada ejercicio se indicará qué se espera visualizar; la elección del método queda a criterio del grupo según el entorno de ejecución utilizado.

Método 1 — Ventana en tiempo real.

```
env = gym.make("LunarLander-v3", render_mode="human")
```

Abre una ventana de pygame que muestra al agente actuando en tiempo real. **Ventajas:** inmediato, sin configuración adicional. **Limitaciones:** solo funciona en ejecución local con pantalla física; no disponible en Google Colab.

Método 2 — Grabación en video (`RecordVideo`).

```
from gymnasium.wrappers import RecordVideo

env = RecordVideo(
    gym.make("LunarLander-v3", render_mode="rgb_array"),
    video_folder="./videos",
    episode_trigger=lambda ep: ep % 100 == 0  # graba cada 100 episodios
)
```

Guarda episodios seleccionados como archivos `.mp4`. **Ventajas:** funciona en Colab y localmente; permite documentar la evolución del agente sin interrumpir el entrenamiento.

Limitaciones: genera archivos en disco que en Colab deben descargarse manualmente al finalizar la sesión.

Método 3 — Animación inline en el notebook.

```python
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

def render_episode(env, agent):
    frames = []
    obs, _ = env.reset()
    done = False
    while not done:
        frames.append(env.render())
        action = agent.select_action(obs)
        obs, _, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
    fig, ax = plt.subplots()
    ax.axis("off")
    img = ax.imshow(frames[0])
    def update(i):
        img.set_data(frames[i])
    anim = FuncAnimation(fig, update, frames=len(frames))
    plt.close()
    return HTML(anim.to_jshtml())

# Requiere render_mode="rgb_array" al crear el entorno
render_episode(env, trained_agent)
```

Muestra la animación directamente en la celda del notebook, sin archivos externos. **Ventajas:** funciona en Colab y Jupyter; ideal como evidencia reproducible. **Limitaciones:** puede ser lento en episodios muy largos; requiere `render_mode="rgb_array"`.

Método 4 — Virtual display en Colab.

```
!pip install pyvirtualdisplay
!apt-get install -y xvfb

from pyvirtualdisplay import Display
Display(visible=False, size=(400, 300)).start()

env = gym.make("LunarLander-v3", render_mode="human")
```

Simula un display en el servidor de Colab para habilitar el Método 1. **Ventajas:** permite usar `render_mode="human"` en Colab. **Limitaciones:** más frágil; puede fallar según la versión del entorno de Colab disponible al momento de ejecución.

2. Ejercicio 2 — Q-Learning clásico: Taxi-v3

Descripción del entorno

Taxi-v3 es un entorno de grilla 5×5 en el que un taxi tiene que subir a un pasajero en una de cuatro posiciones fijas y llevarlo a su destino correcto. El espacio de estados es

discreto y finito; el espacio de acciones incluye seis opciones: moverse en las cuatro direcciones cardinales, subir al pasajero y dejarlo.

Documentación oficial: gymnasium.farama.org — Taxi-v3.

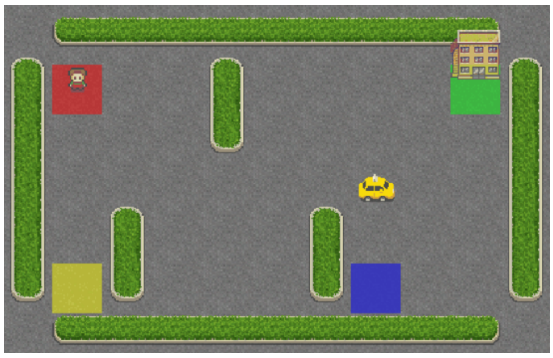


Figura 4: Frame representativo de un episodio del entorno Taxi-v3. El taxi tiene que subir al pasajero y llevarlo al destino indicado navegando una grilla 5×5.

Consignas

2.1. Análisis del MDP.

Identificar y documentar la tupla $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ para este entorno:

a) Indicar el tamaño del espacio de estados $|\mathcal{S}|$ y del espacio de acciones $|\mathcal{A}|$. ¿Cuántas entradas tiene la Q-Table resultante?

b) Describir la función de recompensa que provee el entorno por defecto. ¿En qué momentos y con qué magnitudes se emiten señales de recompensa?

c) Justificar si $P(s'|s, a) = 1$ para todas las transiciones o si existe estocasticidad.

d) Especificar el valor de γ que se utilizará y justificar la elección en función del tipo de tarea.

2.2. Implementación de Q-Learning.

Implementar el algoritmo Q-Learning desde cero **sin utilizar librerías de RL** (no se permite el uso de Stable-Baselines3, RLlib ni similares). La implementación debe incluir:

a) Inicialización de la Q-Table.

b) Política ε-greedy con decaimiento exponencial de ε.

c) Regla de actualización por diferencia temporal (TD):

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big].$$

d) Loop de entrenamiento completo.

2.3. Diseño de la función de recompensa.

Diseñar e implementar **al menos dos funciones de recompensa alternativas** a la que provee el entorno por defecto. Para cada una:

a) Describir la lógica de diseño: ¿qué comportamientos se incentivan o penalizan y por qué?

b) Entrenar el agente bajo esa función y registrar las curvas de aprendizaje.

Graficar las curvas de convergencia de las **tres versiones** (función por defecto + dos alternativas propias) en un mismo gráfico. Analizar cuál converge más rápido, cuál es más estable y justificar las diferencias observadas.

2.4. Análisis de convergencia y visualización.

a) Graficar el reward acumulado por episodio y la tasa de éxito en ventanas de 100 episodios.

b) Reportar el número de episodios necesarios para alcanzar una tasa de éxito ≥ 90 %.

c) Utilizando alguno de los métodos descriptos en la sección *Visualización del agente*, mostrar **al menos un episodio completo** del agente ya entrenado.

3. Ejercicio 3 — Deep Q-Learning (DQN): LunarLander-v3

Descripción del entorno

LunarLander-v3 simula el aterrizaje de una nave espacial en una plataforma. El agente controla tres motores y debe aterrizar suavemente entre las banderas que marcan la zona de destino. El espacio de estados es un **vector continuo de 8 componentes**; el espacio de acciones es discreto con 4 opciones.

Documentación oficial: gymnasium.farama.org — LunarLander.

Instalación: *este entorno requiere las dependencias de Box2D:* `pip install "gymnasium[box2d` *En versiones de Gymnasium* $\geq 1{,}0$ *el identificador* `LunarLander-v2` *fue reemplazado por* `LunarLander-v3`.

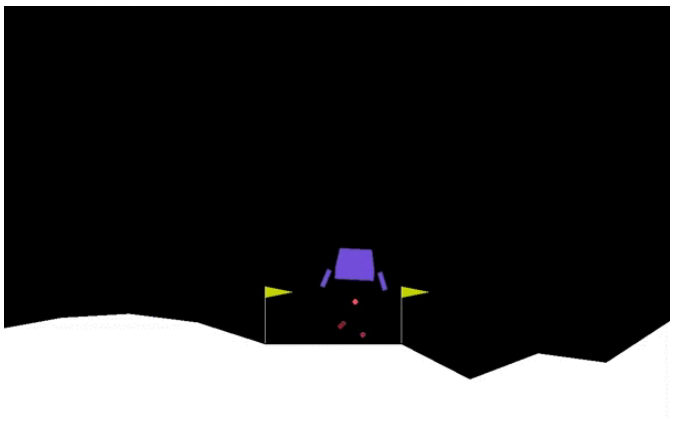


Figura 5: Frame representativo de un episodio del entorno LunarLander. La nave tiene que aterrizar suavemente entre las banderas usando tres motores como acciones disponibles.

Consignas

3.1. Análisis del MDP y motivación del DQN.

Análisis del entorno. Identificar y documentar la tupla ($\mathcal{S}$, $\mathcal{A}$, P, R, γ):

a) Describir los 8 componentes del vector de estado y las 4 acciones disponibles.

b) Identificar qué eventos generan recompensa, en qué magnitud y con qué frecuencia a lo largo de un episodio.

c) Indicar si el entorno es determinístico o estocástico, y si la tarea es episódica o continua.

¿Por qué Q-Learning clásico no es viable acá? El estado de LunarLander-v3 es un vector de 8 valores continuos. Para aplicar Q-Learning clásico sería necesario discretizar ese espacio. Suponer que cada variable se divide en k bins uniformes y responder:

d) ¿Cuántos estados discretos resultarían para $k = 10$? ¿Y para $k = 20$?

e) ¿Cuántas entradas tendría la Q-Table en cada caso (considerando las 4 acciones)?

f) Comparar esos números con las 3.000 entradas de la Q-Table de Taxi-v3 (500 estados $\times$ 6 acciones). ¿Qué concluís sobre la viabilidad del enfoque tabular? Este fenómeno tiene un nombre en la literatura: ¿cuál es y a qué se debe?

g) Con base en esta limitación, explicar por qué reemplazar la Q-Table por una red neuronal es la solución natural y qué ventaja concreta aporta esa sustitución.

3.2. Ablación: impacto de Experience Replay y Target Network.

Esta sección estudia la contribución individual de cada innovación del DQN original. Entrenar **tres versiones del agente** con un límite fijo de **400 episodios** cada una:

Versión	**Experience Replay**	**Target Network**
(a) DQN base	×	×
(b) DQN + ER	✓	×
(c) DQN completo	✓	✓

Las tres versiones deben utilizar **idénticos hiperparámetros**: misma arquitectura de red, tasa de aprendizaje, γ, esquema de decaimiento de ε y tamaño de batch.

Registrar y graficar para cada versión:

a) Reward acumulado por episodio (con media móvil de 20 episodios).

b) Loss de entrenamiento por paso de optimización.

Con base en los gráficos, desarrollar en el informe:

c) ¿En cuál o cuáles versiones se manifiesta el problema del *moving target*? ¿Cómo se evidencia visualmente en la curva de loss?

d) ¿Por qué la correlación temporal entre muestras consecutivas perjudica el entrenamiento en la versión (a), y cómo lo resuelve el Experience Replay en la

versión (b)?

e) ¿Qué agrega la Target Network sobre la versión (b)? ¿Por qué no alcanza con tener solo Experience Replay?

Nota

El objetivo de esta sección es la comparación de **estabilidad**, no la convergencia. El límite de 400 episodios se establece porque las versiones (a) y (b) pueden no converger dentro de ese límite ni fuera de él: en entornos complejos como LunarLander, la ausencia de Experience Replay o Target Network tiende a producir divergencia sostenida, no solo aprendizaje lento. La divergencia en sí es parte del resultado esperado y debe analizarse como tal.

3.3. Entrenamiento completo y visualización.

Entrenar el DQN completo (versión c del punto anterior) hasta alcanzar un reward promedio ≥ 200 en 100 episodios consecutivos, o hasta un máximo razonable de episodios si no se alcanza ese umbral.

a) Graficar el reward por episodio y la media móvil (ventana de 50 episodios).

b) Reportar: episodio en el que el reward promedio cruza por primera vez el valor 0, y episodio de convergencia final.

Visualización de la evolución del agente. Utilizando alguno de los métodos descriptos en la sección *Visualización del agente*, registrar o animar **un episodio completo** en cada uno de los siguientes momentos:

c) **Episodio 0:** agente sin entrenar (comportamiento aleatorio).

d) **Episodio 200:** agente en proceso de aprendizaje.

e) **Episodio de convergencia:** agente entrenado.

Incluir las tres visualizaciones en el notebook. En el informe, describir cualitativamente las diferencias de comportamiento observadas entre los tres momentos: ¿qué aprende primero el agente? ¿Qué errores persisten hasta el final?

3.4. Sensibilidad a hiperparámetros.

Comparar el desempeño del DQN completo bajo **al menos dos configuraciones de hiperparámetros** distintas a la utilizada en el punto anterior. Entre las configuraciones, modificar al menos dos de los siguientes parámetros: tasa de aprendizaje (`lr`), factor de descuento (γ), tamaño del batch (`batch_size`), frecuencia de sincronización de la Target Network.

Para cada configuración reportar:

a) Curva de reward con media móvil y episodio de convergencia, o indicación explícita de que no convergió dentro del límite de episodios establecido.

b) Análisis de cómo cada parámetro modificado afectó la velocidad de convergencia y la estabilidad del entrenamiento.

4. Ejercicio 4 — Análisis teórico comparativo

Este ejercicio no requiere código. Todas las respuestas deben desarrollarse en el **informe escrito**.

4.1. Comparación de métodos.

Completar la siguiente tabla con los resultados obtenidos en los Ejercicios 2 y 3:

	Taxi-v3 · Q-Learning	**LunarLander-v3 · DQN**
Tipo de espacio de estados		
Representación de la política aprendida		
Generalización entre estados similares		
Episodios hasta convergencia		
Tiempo de entrenamiento aproximado		

Agregar un párrafo de síntesis que responda: ¿en qué condiciones es preferible Q-Learning clásico y en cuáles DQN?

4.2. El rol del factor γ.

Tomando los resultados de las consignas 2.1 y 3.4, analizar cómo el factor de descuento γ afecta al agente de forma distinta según el entorno:

a) ¿Qué implica un γ cercano a 1 en Taxi-v3, donde los episodios son cortos y la recompensa final es clara?

b) ¿Cómo cambia esa interpretación en LunarLander-v3, donde la duración de los episodios varía considerablemente entre el inicio y el final del entrenamiento?

4.3. Reflexión final.

Responder las siguientes preguntas con justificación:

a) Se plantea un nuevo problema de RL con un espacio de estados discreto y finito de tamaño 10.000 y 8 acciones posibles. ¿Utilizarías Q-Learning clásico o DQN? ¿De qué otros factores dependería la decisión más allá del tamaño del espacio de estados?

b) DQN resuelve varias limitaciones del Q-Learning clásico, pero introduce restricciones propias. ¿Cuál es la limitación más importante que queda sin resolver en los entornos trabajados en este TP? ¿Qué clase de algoritmos la abordan y cuál es la idea central detrás de ellos?

5. Modalidad de Entrega y Evaluación

El trabajo debe realizarse de manera grupal con exactamente **3 integrantes** (o 4 en caso de que la cantidad de estudiantes del curso no sea múltiplo de 3). Todo el material deberá

almacenarse en un repositorio **GitHub privado**, al cual deberán agregar a los docentes como colaboradores.

Fecha límite de entrega: *viernes 6 de noviembre*, hasta las 23:59 hs. No se aceptará ningún trabajo entregado con posterioridad a dicha fecha y hora.

La entrega se realizará mediante un **correo electrónico con copia a ambos docentes**, con el siguiente formato:

- **Asunto:** `AAII - Entrega TP N°2 - Grupo Y` (reemplazar `Y` por el número del grupo).
- **Cuerpo:** debe incluir:
  1. Número de grupo.
  2. Nombre completo de cada integrante.
  3. Link al repositorio de GitHub.

5.1. Notebooks

El trabajo debe resolverse en **dos Jupyter Notebooks separados**:

- **Notebook 1:** Ejercicio 1 (CRNN).
- **Notebook 2:** Ejercicios 2 y 3, dado que comparten el contexto de implementación. El Ejercicio 4 se responde íntegramente en el informe.

Cada notebook debe ejecutarse completamente de principio a fin sin errores y cumplir los siguientes requisitos:

- El código debe organizarse en celdas según su propósito, con salidas claras (prints y visualizaciones) que documenten el resultado de cada proceso.
- Cada celda de código debe ir precedida de una celda Markdown con: **título** (usando jerarquía `#`, `##`, `###`), una breve descripción de lo que se realizará y las decisiones tomadas con su justificación.
- Cada celda de código debe ir seguida de una celda Markdown que explique el resultado obtenido y las conclusiones derivadas.
- El código debe incluir comentarios explicativos sobre conjuntos de líneas con un mismo propósito. Se deben usar nombres de variables descriptivos en notación `snake_case`.

5.2. Informe

El repositorio deberá contener un **informe en formato PDF** que incluya:

- Descripción del procedimiento seguido para resolver cada ejercicio.
- Respuestas a todas las consignas de análisis de los cuatro ejercicios.
- Decisiones de implementación y de diseño tomadas, con sus justificaciones.
- Gráficos y visualizaciones requeridas (curvas de aprendizaje, tablas de comparación, matrices de confusión, etc.).

- Desafíos encontrados durante el desarrollo.
- Conclusiones sobre los aprendizajes obtenidos.

No incluir bloques de código extensos en el informe; solo fragmentos breves cuando sean necesarios para justificar decisiones de implementación.

5.3. Defensa

La defensa se realizará de manera **virtual** en dos fechas consecutivas. La fecha y hora de cada grupo será sorteada y comunicada con anticipación. El grupo deberá conectarse al enlace de videoconferencia en el horario asignado. **Todos los integrantes deben contar con cámara y mantenerla encendida durante toda la defensa. El integrante que no se presente a la defensa desaprobará el trabajo completo.** De no poder asistir en la fecha u horario sorteado, se puede negociar con otro grupo para intercambiar lugares.

La defensa consta de tres fases:

1. **Exposición grupal** (máximo 20 minutos en total): el grupo deberá presentar el desarrollo de las resoluciones a los ejercicios, las decisiones tomadas en cada uno y sus resultados. Se evaluará la capacidad de seleccionar los aspectos más relevantes y la gestión del tiempo. Se recomienda distribuir la exposición equitativamente entre los integrantes.
2. **Preguntas conceptuales individuales**: cada integrante responderá preguntas de carácter conceptual para terminar de definir la nota.
3. **Devolución general** por parte de los docentes.